QML Lab • research project 16

Simulator vs IBM Hardware Benchmark

Measuring and explaining the gap between simulated and real-device results.

Living document • last updated October 25, 2026

Quadrant	CQ (classical data, quantum processor)	**Track**	Noise and hardware
Difficulty	Medium	**Tools**	Qiskit, Qiskit Runtime, Qiskit Aer
Builds on	Modules 2 and 8 of the QML Lab	**Workspace**	`qml_projects/simulator-vs-ibm-hardware/`

Summary. Every result in the lab so far comes from simulators. This project runs the best small model on a real IBM quantum processor, measures the gap to the simulator, and builds an error budget that explains where the gap comes from: readout errors, gate errors, transpilation and drift. It produces the lab's reference for what the other projects can expect on hardware.

1 Background and key concepts

1.1 From circuit to device

A circuit must be transpiled to the device's native gates and qubit connectivity, which adds SWAP gates and depth. Qiskit (Javadi-Abhari et al., 2024) handles this, and Qiskit Runtime executes circuits through primitives (Sampler and Estimator) with optional error suppression and mitigation. Current devices are noisy intermediate-scale processors without full error correction (Preskill, 2018).

1.2 Error sources

Gate errors
average error rates per gate, measured by randomised benchmarking (Magesan et al., 2011); two-qubit gates dominate.

Readout errors
described by a confusion matrix $A_{ij} = P(\text{read } i \,|\, \text{prepared } j)$; observed distributions are $p_{\text{obs}} = A p_{\text{true}}$ and can be partly inverted (Bravyi et al., 2021).

Decoherence
energy relaxation (T_1) and dephasing (T_2) limit circuit duration.

Drift and crosstalk
calibration changes over hours and days; neighbouring qubits interfere.

1.3 Error budget

The accuracy gap $\Delta = \text{acc}_{\text{sim}} - \text{acc}_{\text{hw}}$ is decomposed by simulating the same circuits with noise models that switch on one error source at a time, using the device's own calibration data. Hardware-efficient circuits (Kandala et al., 2017) keep the transpiled depth low. Kim

et al. (2023) showed that with careful mitigation, current processors can produce accurate expectation values on circuits beyond brute-force simulation, which sets the state of the art for what is achievable.

2 Research questions

The project is open-ended: these questions are the starting point, not the finish line. Add new ones to the README and the journal as they come up.

RQ1 How large is the accuracy gap between simulator and hardware, and how much of it is readout versus gate error?

RQ2 How much does transpilation (layout, optimisation level) change the gap?

RQ3 How stable are hardware results from one day to the next?

3 Method

3.1 Data

The test set of the chosen model (Module 5 or Project 04 classifier with 2–4 qubits, 100 test points); trained weights are fixed beforehand on the simulator. Each run records backend name, date, calibration snapshot, transpiled depth and two-qubit gate count.

3.2 Models

Evaluation pipeline in Qiskit: exact state vector, shot-based simulation, Aer with noise models built from backend calibration (readout only, gates only, full), and the real backend via Runtime with and without readout mitigation. IBM Quantum credentials are read from the local Qiskit account, never stored in the repository.

3.3 Baselines

Exact simulation (upper bound); shot-based simulation with the same number of shots (pure sampling error); Aer with the full device noise model (prediction of hardware performance).

3.4 Metrics

- Test accuracy and expectation-value error against exact values.
- Accuracy gap Δ and its decomposition by error source.
- Transpiled depth, two-qubit gate count and QPU time per run.

4 Planned experiments

Each experiment becomes one `[[experiments]]` entry in the workspace, so its runs, parameters and metrics are stored and compared in the portal.

ID	Experiment	What varies	Main metrics
E1	Sampling only	exact vs shots	accuracy, error
E2	Noise components	readout / gates / full model	predicted Δ
E3	Hardware run	real backend	measured Δ
E4	Mitigation	resilience options	Δ, overhead
E5	Drift	same job on 3 days	spread of results

Table 1: Experiment plan. Results go into the journal and the results log (section 9).

5 Running it in the QML Lab

The workspace lives in `qml_projects/simulator-vs-ibm-hardware/`. Scripts import three helpers from `qml_lab/plots.py`:

- `params()` reads the parameters from the portal's run form (defaults when run by hand);
- `save()` stores figures with the run;
- `log_metrics()` records numbers that appear in the run table.

```
qml_projects/simulator-vs-ibm-hardware/
  project.toml          metadata + experiments
  README.md             goal, open questions, ideas
  experiments/          runnable scripts
  docs/main.tex         this document
```

First experiment: This experiment is *planned*: write the script, add the block below to `project.toml`, then run `python manage.py sync_projects`.

```
[[experiments]]
slug = "hardware-gap"
title = "Simulator vs hardware accuracy for the Module 5 classifier"
script = "experiments/e01_hardware_gap.py"
figure = "gap_breakdown.png"

[experiments.params]
backend = "least_busy"
shots = 4000
optimization_level = 3
mitigate_readout = true
n_test = 100
seed = 0
```

```
# from the portal folder: pick up changes to project.toml
python manage.py sync_projects
cd qml_projects/simulator-vs-ibm-hardware
# run by hand with the default parameters
PYTHONPATH=../../qml_lab python experiments/e01_hardware_gap.py
```

6 Quality bar

- A classical baseline with a similar number of trainable parameters, tuned with the same care (Bowles et al., 2024).
- At least five random seeds per configuration, reported as mean $\pm$ standard deviation.
- How results change with the number of qubits.
- At least one run under a realistic noise model.
- Record backend, date and calibration with every hardware result; without them a number cannot be reproduced.

7 Open directions

- Compare two backends or processor generations.
- Run a quantum kernel (Project 01) and a trained policy (Project 10) on hardware.
- Add zero-noise extrapolation from Project 14.

8 Risks and limitations

- Queue times and free-plan limits restrict the number of runs; plan circuits carefully.
- Calibration drift makes single runs unreliable; repeat across days.
- Never commit API tokens; use the local Qiskit account configuration.

9 Results log

Summarise findings here as they become stable; day-to-day notes belong in the portal journal.

Date	Run	Finding	Next step

References

Bowles, J., Ahmed, S. and Schuld, M. (2024) 'Better than classical? The subtle art of benchmarking quantum machine learning models', Preprint, arXiv:2403.07059. Available at: https://arxiv.org/abs/2403.07059.

Bravyi, S., Sheldon, S., Kandala, A., Mckay, D. C. and Gambetta, J. M. (2021) 'Mitigating measurement errors in multiqubit experiments', *Physical Review A*, 103(4), 042605. Available at: https://doi.org/10.1103/PhysRevA.103.042605.

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J. et al. (2024) 'Quantum computing with Qiskit', Preprint, arXiv:2405.08810. Available at: https://arxiv.org/abs/2405.08810.

Kandala, A., Mezzacapo, A., Temme, K., Takita, M., Brink, M., Chow, J. M. and Gambetta, J. M. (2017) 'Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets', *Nature*, 549, pp. 242–246. Available at: https://doi.org/10.1038/nature23879.

Kim, Y., Eddins, A., Anand, S., Wei, K. X., van den Berg, E. et al. (2023) 'Evidence for the utility of quantum computing before fault tolerance', *Nature*, 618, pp. 500–505. Available at: https://doi.org/10.1038/s41586-023-06096-3.

Magesan, E., Gambetta, J. M. and Emerson, J. (2011) 'Scalable and robust randomized benchmarking of quantum processes', *Physical Review Letters*, 106(18), 180504. Available at: https://doi.org/10.1103/PhysRevLett.106.180504.

Preskill, J. (2018) 'Quantum computing in the NISQ era and beyond', *Quantum*, 2, 79. Available at: https://doi.org/10.22331/q-2018-08-06-79.